

Universidad Fidélitas
Facultad de Ciencias de la Computación

SC-403 Desarrollo de Aplicaciones Web y Patrones

ENUNCIADO DEL PROYECTO FINAL

Contenido

Contenido

Contenido	2
1. Propósito de la actividad	4
2. Pregunta orientadora del proyecto.....	4
3. Resultado esperado	4
4. Organización del equipo y selección del tema.....	5
5. Condiciones técnicas obligatorias.....	6
6. Entregables y valor porcentual.....	6
7. Avance 1 - Semana 5: Historias de usuario y prototipo.....	7
7.1 Objetivo del avance	7
7.2 Productos por entregar	7
7.3 Recomendaciones para historias de usuario	8
7.4 Criterios de evaluación del Avance 1	8
8. Avance 2 - Semana 9: Implementación funcional del 50%	9
8.1 Objetivo del avance	9
8.2 Alcance mínimo esperado	9
8.3 Condiciones de validez del Avance 2.....	9
8.4 Criterios de evaluación del Avance 2	10
9. Artículo científico - Semana 15	10
9.1 Propósito del artículo	10
9.2 Formato y extensión	10
9.3 Estructura mínima del artículo.....	11
9.4 Criterios de evaluación del artículo	11
10. Entrega final del proyecto - Semana 15.....	12
10.1 Objetivo de la entrega final.....	12
10.2 Evidencias obligatorias de entrega	12

10.3 Requisitos funcionales mínimos	12
10.4 Temas del curso que pueden integrarse	13
11. Defensa final - Semana 15	14
11.1 Propósito de la defensa	14
11.2 Estructura sugerida de la defensa	14
12. Rúbrica de la entrega final y defensa	14
13. Condiciones de calificación cero aplicables a cualquier avance del Proyecto Final	16
13.1 Originalidad, trazabilidad y uso de trabajos previos	16
13.2 Tecnologías, dependencias y forma de implementación vistas en el curso	17
13.3 Arquitectura, patrón MVC y estructura del proyecto	17
13.4 Uso de base de datos, datos dinámicos y transaccionalidad	17
13.5 Gestión de imágenes, archivos y recursos	18
13.6 Reutilización indebida del proyecto Tienda	18
13.7 Evidencia de colaboración en GitHub	18
13.8 Defensa, ejecución y verificación técnica	19
13.9 Otras condiciones aplicables	19
14. Recomendaciones para gestión del proyecto	20
14.1 Aclaración sobre el uso de GitHub en el Proyecto Final	20
14.2 Organización inicial del repositorio	20
14.3 Flujo colaborativo esperado	20
14.4 Evidencias esperadas por avance	21
14.5 Prácticas no aceptadas o insuficientes	21
14.6 Recomendaciones de calidad para el repositorio	22
15. Lista de verificación por entrega	23
16. Formato de entrega sugerido	23
17. Honestidad académica y uso responsable de apoyo externo	24
18. Cierre	24

1. Propósito de la actividad

El proyecto final consiste en diseñar, construir, documentar y defender una aplicación web transaccional que responda a una necesidad real o verosímil de una organización, PYME, ONG, emprendimiento, área interna o cliente potencial. La solución debe aplicar de forma integrada los conocimientos desarrollados en el curso: HTML5, CSS, Spring Boot, Thymeleaf, Bootstrap, Hibernate/JPA, bases de datos, arquitectura MVC, autenticación, autorización por roles, internacionalización, componentes transaccionales, uso colaborativo de GitHub y patrones de diseño.

La intención académica es que el estudiantado demuestre que puede transformar requerimientos en un producto web funcional, visualmente claro, intuitivo para los usuarios, técnicamente consistente y útil para la mejora de una organización. El proyecto no se limita a reproducir prácticas de laboratorio: debe evidenciar apropiación, adaptación y ampliación de los temas vistos en clase.

2. Pregunta orientadora del proyecto

¿Cómo diseñar e implementar una solución web transaccional con Java + Spring Boot que mejore un proceso organizacional y la experiencia de sus usuarios, aplicando patrones, persistencia, seguridad, diseño visual e integración de tecnologías web?

3. Resultado esperado

Al finalizar el proyecto, cada equipo deberá entregar y defender una aplicación web funcional que incluya, como mínimo:

- Resolver un problema o necesidad claramente delimitada, preferiblemente vinculada con una organización real, PYME, ONG o cliente potencial.
- Una solución web desarrollada en Java con Spring Boot, siguiendo la estructura de codificación trabajada durante el curso.
- Interfaz web construida con HTML5, Thymeleaf y Bootstrap, con diseño consistente, navegación clara y experiencia de usuario intuitiva.
- Persistencia de datos mediante Hibernate/JPA y base de datos relacional.
- Módulos transaccionales que registren operaciones reales del negocio o proceso elegido.
- Autenticación, autorización y diferenciación de funcionalidades por roles de usuario.

- Internacionalización mediante archivos de idioma.
- Uso colaborativo, verificable y distribuido de GitHub.
- Documentación técnica y académica mediante artículo científico en formato IEEE.
- Defensa oral con demostración funcional, explicación técnica y reflexión sobre resultados.

4. Organización del equipo y selección del tema

El proyecto se desarrolla en equipos de hasta 4 estudiantes. La conformación d los grupos se realiza desde semana 1 en el campus y esto se cierra en semana3, los estudiantes que no conforman grupo, el profesor los agrupa sin tener que consultarles a los estudiantes.

El proyecto es de libre selección, sin embargo, debe permitir la construcción de una aplicación web transaccional. Se recomienda seleccionar procesos donde existan usuarios, roles, datos persistentes, operaciones registrables y reglas de negocio claras. Algunos ejemplos válidos son:

- Gestión de inventario, ventas y pedidos para una PYME.
- Sistema de reservas, citas o inscripción a actividades.
- Control de donaciones, voluntariado o servicios para una ONG.
- Plataforma de comercio electrónico con carrito de compras.
- Gestión de solicitudes, trámites o incidencias en un departamento interno.
- Sistema académico, deportivo, cultural o comunitario con administración de usuarios y transacciones.

No se aceptan proyectos meramente informativos, páginas estáticas, aplicaciones de consola, formularios de escritorio ni desarrollos en lenguajes diferentes a Java para el componente principal del proyecto.

5. Condiciones técnicas obligatorias

Condición	Requisito mínimo
Lenguaje y framework	Java + Spring Boot. El proyecto debe ejecutarse como aplicación web; no se acepta consola, aplicación de escritorio ni otro lenguaje principal.
Estructura de desarrollo	Debe respetar la estructura y estilo de codificación desarrollados durante el curso: modelo MVC, controladores, servicios, repositorios, entidades y vistas.
Interfaz	Uso obligatorio de Bootstrap según lo trabajado en clase. La ausencia de Bootstrap invalida el avance o entrega correspondiente, según lo establecido en el programa. Igualmente incorporar Bootstrap de otra manera no vista en clase invalida el proyecto.
Motor de plantillas	Uso de Thymeleaf, fragmentos, plantillas y archivos de idioma.
Persistencia	Uso de Hibernate/JPA con base de datos relacional. En la entrega final se requiere al menos 8 tablas y al menos una tabla destinada a registrar transacciones.
Autenticación y roles	Debe existir inicio de sesión, roles definidos y restricción efectiva de accesos/funcionalidades según el rol.
Temas del curso	El proyecto debe integrar al menos 6 temas vistos en clase, además del almacenamiento de información.
Investigación adicional	Debe incorporarse al menos una funcionalidad o tecnología no desarrollada directamente en clase, con explicación de su aporte al sistema.
GitHub	Repositorio activo con participación de todos los miembros. Se espera evidencia de aportes significativos mediante commits, ramas o pull requests.
Documentación	README técnico, instrucciones de ejecución, scripts o respaldos de base de datos y artículo científico en formato IEEE.

6. Entregables y valor porcentual

Semana	Entregable	Producto principal	Valor
5	Avance 1	Historias de usuario y diseño de prototipo.	5%
9	Avance 2	Avance funcional del 50% del proyecto, fiel al prototipo y a la estructura del curso.	10%
15	Artículo científico	Informe escrito en formato IEEE sobre el proceso de construcción, resultados y discusión.	10%
15	Entrega final y defensa	Aplicación web transaccional completa, evidencias técnicas, repositorio y presentación oral.	25%
	Total, proyecto final integrado	Componente técnico 40% + artículo 10%.	50%

7. Avance 1 - Semana 5: Historias de usuario y prototipo

Valor: 5% de la nota final del curso.

7.1 Objetivo del avance

Definir con claridad el producto que se construirá, sus usuarios, funcionalidades principales, reglas de negocio iniciales y diseño visual preliminar, antes de iniciar la codificación intensiva.

7.2 Productos por entregar

- Documento de planteamiento del proyecto: nombre de la solución, descripción del problema, cliente real o potencial, usuarios meta y justificación.
- Al menos 20 historias de usuario redactadas de forma específica, alineadas con la necesidad del proyecto.
- Criterios de aceptación para cada historia de usuario o, como mínimo, para las historias prioritarias.
- Priorización inicial del backlog: alta, media y baja prioridad.
- Prototipo navegable o conjunto de pantallas principales elaborado con una herramienta adecuada. Puede utilizarse Figma, Balsamiq, draw.io, Canva, PowerPoint u otra herramienta autorizada por el docente.
- Mapa de navegación o flujo de pantallas que muestre cómo interactúan los usuarios con la aplicación.
- Modelo preliminar de datos o listado de entidades principales previstas.
- Repositorio GitHub inicial creado para el equipo, con los integrantes agregados como colaboradores, README preliminar y acuerdo básico de trabajo por ramas.
- Presentación breve mediante vídeo, con participación de todos los integrantes.

7.3 Recomendaciones para historias de usuario

Se sugiere redactar cada historia con la estructura: Como [rol de usuario], necesito [funcionalidad], para [beneficio o finalidad].

Elemento	Ejemplo
Rol	Administrador, cliente, vendedor, encargado, colaborador, voluntario, usuario visitante.
Funcionalidad	Registrar productos, gestionar pedidos, aprobar solicitudes, consultar historial, generar reporte.
Finalidad	Controlar inventario, agilizar atención, dar seguimiento, mejorar experiencia del usuario, tomar decisiones.

7.4 Criterios de evaluación del Avance 1

Criterio	Peso interno	Nivel estratégico / excelente	Evidencia esperada
Participación de miembros	20%	Todos los integrantes participan de forma equitativa, dominan la propuesta y pueden justificar decisiones.	Presentación grabada, distribución clara de intervenciones.
Historias de usuario	40%	Se presentan al menos 20 historias específicas, alineadas al problema, con roles claros y criterios de aceptación pertinentes.	Documento de historias y backlog priorizado.
Prototipo	30%	El prototipo representa adecuadamente las funcionalidades clave, es navegable o comprensible, y coincide con las historias de usuario.	Archivo o enlace del prototipo, mapa de navegación.
Viabilidad y coherencia técnica	10%	La propuesta es realizable en el tiempo del curso y permite aplicar Spring Boot, base de datos, roles, Bootstrap y componentes transaccionales.	Explicación técnica inicial, entidades y alcance delimitado.

8. Avance 2 - Semana 9: Implementación funcional del 50%

Valor: 10% de la nota final del curso.

8.1 Objetivo del avance

Demostrar que el equipo ya transformó el diseño inicial en una aplicación web funcional, con avance verificable de codificación, persistencia, navegación, diseño Bootstrap y participación colaborativa en GitHub.

8.2 Alcance mínimo esperado

- Al menos 50% de las historias de usuario implementadas o, como mínimo, las historias de mayor prioridad con flujo funcional completo.
- Proyecto Spring Boot organizado en capas: Controller, Service, Repository, Domain y vistas Thymeleaf.
- Uso efectivo de Bootstrap en las vistas desarrolladas.
- Conexión a base de datos funcional mediante Hibernate/JPA.
- CRUD implementado para entidades principales.
- Navegación coherente con el prototipo de Avance 1.
- Repositorio GitHub actualizado, con aportes significativos de todos los integrantes.
- Evidencia de trabajo colaborativo mediante ramas funcionales, pull requests, revisión de cambios y aceptación controlada hacia la rama principal.
- README preliminar con descripción, requisitos de ejecución y estado del avance.
- Demostración del sistema ante el docente, evidenciando funcionamiento real y no únicamente pantallas estáticas.

8.3 Condiciones de validez del Avance 2

La codificación debe ser fiel a lo visto en el curso. Según el programa, se consignan cero absolutos cuando el avance no utiliza Bootstrap conforme a lo trabajado en clase o cuando usa una estructura/estilo de codificación diferente al desarrollado durante el periodo. También se invalida si no corresponde a una aplicación web en Java + Spring Boot.

8.4 Criterios de evaluación del Avance 2

Criterio	Peso interno	Nivel estratégico / excelente	Evidencia esperada
Participación de miembros	15%	Todos los integrantes participan, explican aportes propios y responden consultas técnicas autoelaboradas.	Defensa del avance y registro de contribuciones.
Implementación de historias de usuario	25%	Se implementa al menos 50% de las historias totales y se priorizan las funcionalidades centrales del negocio.	Backlog marcado, demostración funcional en video.
Funcionalidad y fidelidad al prototipo	30%	El sistema ejecuta flujos reales, persiste datos y respeta o mejora el diseño planteado en el prototipo.	Aplicación ejecutándose, base de datos y vistas Bootstrap.
Participación en GitHub	30%	El 100% de los miembros registra aportes significativos y trazables en el repositorio.	Historial de commits, ramas o pull requests; evidencias de colaboración.

9. Artículo científico - Semana 15

Valor: 10% de la nota final del curso.

9.1 Propósito del artículo

El artículo documenta el proceso de análisis, diseño, construcción, pruebas, resultados y reflexión del proyecto. No es un manual de usuario ni una simple descripción de pantallas; debe explicar cómo se desarrolló la solución, qué decisiones técnicas se tomaron, cómo se aplicaron los contenidos del curso y qué resultados se obtuvieron.

9.2 Formato y extensión

- Extensión sugerida: entre 6 y 12 páginas.
- Formato: plantilla IEEE para artículos científicos.
- Estilo de citación y referencias: IEEE.
- Mínimo recomendado: 10 referencias pertinentes y correctamente citadas dentro del artículo.
- Debe incorporar referencias técnicas, académicas o profesionales relacionadas con Spring Boot, desarrollo web, patrones, bases de datos, seguridad, UX o el dominio del problema.
- El artículo se entrega en semana 15 y se utiliza como parte de la defensa final.

9.3 Estructura mínima del artículo

Sección	Contenido esperado
Título, autores y afiliación	Nombre del proyecto, integrantes, curso y universidad.
Resumen	Síntesis del problema, objetivo, metodología, solución y resultados principales.
Palabras clave	Entre 4 y 6 términos: Spring Boot, MVC, Hibernate, web transaccional, roles, etc.
Introducción	Contexto, problema, justificación, objetivo del proyecto y alcance.
Marco o fundamento técnico	Conceptos utilizados: arquitectura MVC, patrones, persistencia, seguridad, Bootstrap, Thymeleaf, entre otros.
Metodología	Proceso de trabajo, levantamiento de historias de usuario, organización del equipo, Scrum o estrategia de seguimiento, uso de GitHub.
Diseño y desarrollo de la solución	Arquitectura, módulos, roles, base de datos, decisiones de interfaz y tecnologías utilizadas.
Resultados y discusión	Qué implementa el sitio web, qué problemas resuelve, limitaciones, pruebas realizadas y oportunidades de mejora.
Conclusiones y recomendaciones	Aprendizajes técnicos y colaborativos, valor para la organización o cliente potencial.
Referencias	Fuentes citadas en formato IEEE. Evitar referencias no consultadas o no citadas en el texto.

9.4 Criterios de evaluación del artículo

Criterio	Peso interno	Excelente	Evidencia esperada
Resumen e introducción	15%	Presentan con claridad el problema, objetivo, alcance, justificación y aporte del proyecto.	Secciones completas, redacción académica y formato IEEE.
Metodología	20%	Describe de forma ordenada el proceso de creación del sitio, roles del equipo, uso de GitHub y metodología de trabajo.	Explicación del proceso, apoyada en evidencias del repositorio y gestión del proyecto.
Resultados y discusión	25%	Detalla lo implementado, analiza resultados, muestra capturas o evidencias pertinentes y discute limitaciones.	Descripción técnica y funcional del producto final.
Conclusiones y recomendaciones	15%	Presenta reflexión relevante, aprendizajes, mejoras futuras y valor de la solución.	Conclusiones conectadas con los objetivos del proyecto.
Referencias IEEE	25%	Incluye 10 o más referencias pertinentes, citadas en el texto y formateadas correctamente.	Lista de referencias y citas internas en estilo IEEE.

10. Entrega final del proyecto - Semana 15

Valor: 25% de la nota final del curso.

10.1 Objetivo de la entrega final

Presentar una aplicación web transaccional completa, funcional y defendible, que demuestre integración de contenidos del curso, resolución de una necesidad organizacional y dominio técnico del equipo.

10.2 Evidencias obligatorias de entrega

- Repositorio GitHub definitivo, con historial de contribuciones de todos los miembros.
- Historial de ramas, commits y pull requests que demuestre colaboración real, revisión entre pares y participación técnica de todos los integrantes.
- Código fuente completo del proyecto Java + Spring Boot.
- Archivo README con instrucciones de instalación, configuración, ejecución, usuarios de prueba y descripción de módulos.
- Script SQL, respaldo o instrucciones claras para crear y poblar la base de datos.
- Aplicación ejecutable localmente o desplegada en un servidor/plataforma autorizada por el docente.
- Credenciales de prueba para cada rol implementado.
- Documento o enlace del artículo científico en formato IEEE.
- Presentación de defensa y demostración funcional.
- Evidencia de participación del cliente real/potencial o análisis de oferta de mercado cuando no exista cliente directo.

10.3 Requisitos funcionales mínimos

- Base de datos con al menos 8 tablas.
- Al menos una tabla debe registrar transacciones del sistema, por ejemplo: pedidos, reservas, solicitudes, compras, movimientos, citas, inscripciones, pagos simulados o registros equivalentes.
- Módulo de autenticación y autorización con roles reales.
- Restricción de accesos por rol en menús, vistas o acciones.
- Uso medular de la base de datos; la persistencia no debe ser decorativa.
- Internacionalización integrada de forma funcional.
- Diseño final coherente con el prototipo o mejorado justificadamente.

- Integración de al menos 6 temas desarrollados en clase, además del almacenamiento de información.
- Uso de una temática, librería, funcionalidad o tecnología investigada por el equipo y no desarrollada directamente en clase.
- Aportes colaborativos verificables de todos los miembros en GitHub.
- Evidencia de que la aplicación atiende una necesidad real, un cliente potencial o un análisis fundamentado del mercado.

10.4 Temas del curso que pueden integrarse

Tema	Aplicación posible en el proyecto
HTML5 y CSS	Estructura semántica, formularios, navegación, presentación visual.
Spring Boot	Configuración del proyecto, controladores, rutas, manejo de solicitudes.
Thymeleaf	Vistas dinámicas, plantillas, fragmentos y uso de expresiones.
Bootstrap	Diseño responsive, componentes, tablas, formularios, modales y estilos consistentes.
Hibernate/JPA	Entidades, repositorios, relaciones, persistencia y consultas.
Modelo MVC	Separación clara entre vista, controlador, modelo, servicios y repositorios.
CRUD	Mantenimiento de entidades principales del negocio.
Relaciones	Asociaciones muchos a uno, uno a muchos y consultas relacionadas.
Consultas JPQL/SQL	Búsquedas, filtros, reportes o consultas ampliadas.
Firebase Storage	Almacenamiento de imágenes o archivos vinculados al sistema, cuando aplique.
Autenticación y roles	Acceso diferenciado según administrador, cliente, encargado u otros roles.
Spring Mail	Notificaciones o confirmaciones por correo, cuando aplique.
Carrito de compras	Componente comercial o transaccional adaptable al dominio elegido.
Servicios web/microservicios	Consumo o exposición básica de servicios, si el alcance del proyecto lo permite.
Patrones de diseño	Aplicación justificada de buenas prácticas de diseño y organización del código.

11. Defensa final - Semana 15

11.1 Propósito de la defensa

La defensa permite verificar la autoría, dominio técnico, calidad funcional y capacidad de comunicación del equipo. Cada integrante debe explicar el sistema, demostrar su aporte y responder preguntas del docente.

11.2 Estructura sugerida de la defensa

- Contexto del problema: cliente, necesidad, usuarios y objetivo de la solución.
- Recorrido por el artículo científico: resumen del proceso, metodología, resultados y conclusiones.
- Arquitectura técnica: capas, entidades, base de datos, roles, tecnologías y patrones utilizados.
- Demostración funcional: ingreso al sistema, operaciones principales, transacciones, roles, internacionalización y funcionalidades investigadas.
- Evidencia de colaboración: GitHub, distribución de tareas, decisiones tomadas y aprendizajes.
- Explicación del flujo colaborativo utilizado en GitHub: creación de ramas, pull requests, revisión, resolución de conflictos y unión de cambios a la rama principal.
- Cierre reflexivo: limitaciones, mejoras futuras y valor del proyecto para la organización o cliente potencial.

Durante la defensa, el docente puede solicitar navegar por cualquier módulo, revisar el código fuente, verificar la base de datos, consultar el historial de GitHub o pedir explicación individual a cualquier integrante.

12. Rúbrica de la entrega final y defensa

Criterio	Peso interno	Receptivo	Resolutivo	Estratégico
Almacenamiento	10%	Menos de 8 tablas o estructura insuficiente.	8 tablas o más, pero sin tabla transaccional clara.	8 tablas o más y al menos una tabla registra transacciones reales del sistema.

Autenticación y roles	10%	No existe autenticación o los roles no están definidos.	Existe autenticación y roles, pero no limitan accesos de forma efectiva.	Autenticación y roles controlan correctamente menús, vistas y acciones.
Uso efectivo de base de datos	10%	La base de datos tiene uso secundario o decorativo.	La base de datos soporta parte del sistema, pero con limitaciones.	La base de datos cumple una función modular en el sistema.
Internacionalización	8%	No está presente o no funciona.	Está presente, pero con uso parcial.	Está integrada de forma funcional y visible en la experiencia del usuario.
Diseño final	10%	Difiere sensiblemente del prototipo o presenta baja usabilidad.	Es similar al prototipo, aunque faltan detalles.	Respeto o mejora el prototipo, con interfaz clara, consistente y usable.
Uso de temáticas del curso	15%	Implementa menos del 80% de los temas esperados o los usa de forma superficial.	Adapta varios temas, aunque con integración limitada.	Integra apropiadamente al menos 6 temas del curso, con apropiación técnica evidente.
Investigación adicional	7%	La funcionalidad investigada no existe o no aporta al sistema.	La funcionalidad existe, pero tiene aporte parcial.	La funcionalidad investigada tiene función modular o valor claro para la solución.
Uso colaborativo de GitHub	10%	La participación es desigual o insuficiente.	Hay aportes de varios integrantes, pero no todos evidencian	Todos los integrantes registran aportes significativos, distribuidos y verificables.

			participación consistente.	
Solución real o cliente potencial	10%	El proyecto es imaginario o sin necesidad demostrable.	Se declara una necesidad, pero con poca evidencia.	Existe evidencia de cliente, necesidad real o análisis de mercado pertinente.
Estilo de presentación y defensa	10%	La presentación omite la navegación por el artículo o evidencia poco dominio.	Presenta el artículo y el sistema, aunque con explicación parcial.	Integra artículo, demostración, decisiones técnicas y participación equitativa.

13. Condiciones de calificación cero aplicables a cualquier avance del Proyecto Final

Las siguientes condiciones aplican al Avance 1 de semana 5, Avance 2 de semana 9, artículo científico, entrega final del proyecto y defensa de semana 15, según la naturaleza de cada producto. El incumplimiento de cualquiera de estas condiciones podrá implicar una calificación de cero en el avance o entregable correspondiente, aun cuando el sistema ejecute parcialmente o presente una apariencia visual aceptable. El objetivo de esta sección es resguardar la originalidad del trabajo, la trazabilidad del proceso y la evidencia de aprendizaje de los contenidos desarrollados durante el curso.

13.1 Originalidad, trazabilidad y uso de trabajos previos

- Utilizar, total o parcialmente, un proyecto elaborado en un periodo anterior, independientemente del docente que impartió el curso, del grupo en que se desarrolló o de que el estudiante haya participado anteriormente en su construcción.
- Presentar como propio un proyecto, módulo, repositorio, plantilla, base de datos o documentación desarrollada por otra persona, equipo, curso, institución o fuente externa sin autorización expresa y sin la debida trazabilidad académica.
- Reutilizar código, estructura, documentación o recursos de cuatrimestres anteriores con cambios superficiales de nombres, colores, imágenes, textos o menús, sin evidenciar un desarrollo nuevo y progresivo durante el cuatrimestre actual.

- Entregar avances sin historial verificable de construcción progresiva en GitHub, por ejemplo, repositorios creados al final del periodo, cargas masivas de archivos, commits no significativos o ausencia de participación real de los integrantes.

13.2 Tecnologías, dependencias y forma de implementación vistas en el curso

- No desarrollar el proyecto como una aplicación web transaccional en Java con Spring Boot, conforme a la naturaleza técnica del curso.
- No utilizar las dependencias, configuración y forma de inclusión de Bootstrap, Thymeleaf, JPA/Hibernate, controladores, modelos, repositorios, servicios y demás componentes según lo trabajado en clase, aunque la solución aparentemente funcione.
- Sustituir las tecnologías obligatorias del curso por otras no autorizadas, tales como aplicaciones de consola, aplicaciones de escritorio, sitios estáticos, frameworks distintos o lenguajes diferentes para el desarrollo principal del proyecto.
- Incorporar Bootstrap, Thymeleaf, JPA u otras librerías mediante estructuras, rutas, dependencias o patrones de codificación no trabajados durante el periodo, cuando esto impida verificar la apropiación de los aprendizajes esperados.
- Presentar una solución que dependa de código generado o plantillas externas que no puedan ser explicadas, modificadas o defendidas técnicamente por el equipo.

13.3 Arquitectura, patrón MVC y estructura del proyecto

- Generar un esquema o estructura de proyecto no vista en el curso, especialmente si no se aplica el patrón Modelo-Vista-Controlador (MVC).
- No separar adecuadamente las responsabilidades entre entidades/modelos, repositorios, servicios, controladores, plantillas y recursos estáticos.
- Concentrar la lógica del negocio, el acceso a datos o el flujo de navegación en lugares no correspondientes, por ejemplo, lógica modular dentro de vistas HTML, controladores sin uso de servicios o páginas desconectadas de la capa de datos.
- Presentar una estructura de carpetas, paquetes, nombres o clases que no permita reconocer la organización técnica enseñada durante el curso.

13.4 Uso de base de datos, datos dinámicos y transaccionalidad

- No utilizar base de datos para generar las páginas principales del sistema, es decir, construir páginas estáticas con contenido HTML quemado en lugar de información consultada, registrada, actualizada o eliminada desde la base de datos.

- Presentar funcionalidades que simulan operaciones transaccionales, pero que no almacenan ni recuperan información real mediante JPA/Hibernate y una base de datos configurada.
- No evidenciar que la base de datos cumple una función modular en el sistema desarrollado.
- Usar la misma información de base de datos del proyecto Tienda o datos de ejemplo copiados sin adaptación al contexto, cliente potencial, dominio funcional o problema seleccionado por el equipo.
- Usar los mismos usuarios, roles, credenciales, perfiles o combinaciones de prueba del proyecto Tienda, sin responder a los actores reales o simulados del nuevo proyecto.
- No incluir una estructura de datos coherente con la naturaleza transaccional del proyecto final, incluyendo tablas, relaciones y registros propios del dominio seleccionado.

13.5 Gestión de imágenes, archivos y recursos

- Usar imágenes locales como base funcional del proyecto cuando, según lo trabajado en clase, corresponda utilizar almacenamiento en la nube o mecanismos equivalentes autorizados por el docente.
- Incluir recursos gráficos, archivos o enlaces que funcionen únicamente en la computadora de un integrante y no puedan ser ejecutados, revisados o demostrados desde otro entorno.
- Presentar rutas absolutas locales, dependencias manuales o archivos no versionados que impidan la ejecución del proyecto por parte del docente.

13.6 Reutilización indebida del proyecto Tienda

- Utilizar la misma estructura exacta del proyecto Tienda como base del Proyecto Final, sin rediseño funcional, técnico y contextual acorde con el problema seleccionado.
- Reproducir la misma estructura de menú del proyecto Tienda, incluyendo opciones, jerarquías, nombres o flujos de navegación equivalentes sin justificación funcional propia.
- Mantener las mismas entidades, tablas, relaciones, datos, roles, usuarios, pantallas o procesos del proyecto Tienda, aun cuando se cambien textos, colores o imágenes.
- Presentar el Proyecto Final como una copia adaptada del proyecto Tienda, en lugar de una solución nueva, colaborativa, contextualizada y alineada con al menos seis temas desarrollados durante el curso.

13.7 Evidencia de colaboración en GitHub

- No trabajar el Proyecto Final mediante un repositorio colaborativo en GitHub con integrantes agregados como colaboradores.

- No evidenciar el uso de ramas, commits significativos, pull requests, revisión de cambios y aceptación hacia la rama principal.
- Concentrar el desarrollo en una sola cuenta, realizar commits en nombre de otros integrantes o simular participación colaborativa sin aportes técnicos verificables.
- No poder explicar durante la defensa qué aportó cada integrante, qué ramas trabajó, qué pull requests realizó o revisó y cómo se integraron los cambios al proyecto.

13.8 Defensa, ejecución y verificación técnica

- No poder ejecutar o demostrar el sistema durante la defensa, salvo justificación técnica aceptada por el docente y evidencia suficiente del funcionamiento previo.
- No poder explicar el código fuente, la estructura del proyecto, la configuración de dependencias, el modelo de datos, la autenticación, los roles o las funcionalidades transaccionales desarrolladas.
- Presentar una defensa donde los integrantes no demuestren dominio proporcional del proyecto o no puedan responder preguntas técnicas sobre los módulos que declararon haber desarrollado.
- Omitir evidencias solicitadas en Moodle, enlaces de repositorio, scripts de base de datos, credenciales de prueba, documentación de instalación, artículo científico o recursos necesarios para la revisión.

13.9 Otras condiciones aplicables

Cualquier evidencia de fraude académico, plagio, falsificación de autoría, manipulación del historial de GitHub, uso no autorizado de trabajos ajenos o incumplimiento de las instrucciones publicadas en este documento podrá implicar la calificación de cero del entregable correspondiente y la aplicación de la normativa institucional vigente.

El docente podrá valorar otros incumplimientos equivalentes no listados de forma expresa en esta sección, siempre que afecten la originalidad, la trazabilidad, la coherencia técnica, la aplicación de los contenidos del curso o la posibilidad de evaluar objetivamente el aprendizaje del equipo.

14. Recomendaciones para gestión del proyecto

14.1 Aclaración sobre el uso de GitHub en el Proyecto Final

El uso de GitHub en la solución del Proyecto Final **no** debe entenderse de la misma manera que en el proyecto individual denominado “Tienda”. En el proyecto Tienda, cada estudiante programa de forma individual, semana a semana, siguiendo el desarrollo realizado en el curso. En cambio, el Proyecto Final es una actividad colaborativa; por tanto, el repositorio, las decisiones técnicas, la integración de código y las evidencias de participación deben reflejar trabajo en equipo real y verificable.

Para este proyecto, GitHub no se utiliza únicamente como espacio para almacenar el código terminado, sino como herramienta de gestión colaborativa del desarrollo. La evidencia esperada debe mostrar que los integrantes diseñan, programan, revisan, integran y documentan la solución mediante un flujo de trabajo compartido.

14.2 Organización inicial del repositorio

Desde las primeras semanas del proyecto, uno de los integrantes debe crear el repositorio oficial del equipo en GitHub y agregar a sus compañeros como colaboradores. El repositorio debe corresponder a una única solución grupal; no se aceptará que cada estudiante trabaje en repositorios aislados y que al final se unan archivos de manera manual.

El repositorio debe contener, como mínimo:

- Nombre claro del proyecto y descripción breve de la solución.
- README inicial con objetivo, integrantes, tecnologías previstas y pasos preliminares de ejecución.
- Estructura base del proyecto Spring Boot conforme a lo trabajado en el curso.
- Lista inicial de tareas, historias de usuario o issues asociados al backlog del proyecto.
- Colaboradores correctamente agregados, de manera que los aportes de cada integrante queden registrados con su usuario de GitHub.

14.3 Flujo colaborativo esperado

La dinámica mínima recomendada para el desarrollo colaborativo es la siguiente:

1. Mantener una rama principal, preferiblemente main, que represente la versión estable o integrable del proyecto.

2. Crear ramas de trabajo por funcionalidad, corrección o mejora, por ejemplo: feature/autenticacion-roles, feature/carrito-compras, fix/validacion-formulario o feature/internacionalizacion.
3. Realizar commits significativos y descriptivos en la rama correspondiente. Los mensajes deben permitir comprender qué se agregó, corrigió o modificó.
4. Publicar la rama en GitHub y crear un pull request hacia la rama principal cuando la funcionalidad esté lista para ser revisada.
5. Solicitar la revisión de, al menos, otro integrante del equipo antes de aceptar el cambio. La revisión puede incluir lectura de código, prueba funcional, validación de estilos, verificación de base de datos o revisión de integración con otras partes del sistema.
6. Atender observaciones, corregir conflictos o ajustar el código antes de unir los cambios a la rama principal.
7. Aceptar el pull request y actualizar la rama principal únicamente cuando el cambio sea funcional, coherente con el proyecto y no afecte negativamente otras funcionalidades.
8. Sincronizar periódicamente los entornos locales de trabajo mediante pull desde la rama principal, para evitar divergencias, duplicidad de código o conflictos acumulados.

14.4 Evidencias esperadas por avance

- **Avance 1, semana 5:** el equipo debe mostrar el repositorio creado, los colaboradores agregados, el README preliminar, la estructura inicial del proyecto o preparación técnica, y la relación entre historias de usuario, tareas o issues.
- **Avance 2, semana 9:** el equipo debe evidenciar que el avance funcional no fue construido por una sola persona. Se espera historial de commits, ramas de trabajo, pull requests y participación técnica de todos los integrantes.
- **Entrega final, semana 15:** el repositorio debe permitir reconstruir el proceso de desarrollo del proyecto. Deben observarse aportes distribuidos, pull requests significativos, integración progresiva de funcionalidades, resolución de conflictos cuando existan y documentación suficiente para ejecutar la aplicación.
- **Defensa final, semana 15:** el equipo debe estar preparado para explicar cómo se organizó el trabajo en GitHub, qué aportó cada integrante, cómo se revisaron los cambios y cómo se aseguró la estabilidad de la rama principal.

14.5 Prácticas no aceptadas o insuficientes

Para efectos de evaluación, no se consideran evidencias colaborativas suficientes las siguientes prácticas:

- Subir el proyecto completo al final del curso sin historial progresivo de trabajo.

- Realizar todos los commits desde la cuenta de una sola persona, aunque el equipo indique que todos participaron.
- Compartir archivos por medios externos y pegarlos manualmente en el repositorio sin trazabilidad.
- Trabajar directamente sobre la rama principal durante todo el proyecto, sin ramas ni pull requests, cuando esto impida verificar revisión e integración colaborativa.
- Crear commits vacíos, superficiales o únicamente de formato para simular participación.
- Aceptar cambios sin revisión cuando estos afecten módulos críticos como seguridad, base de datos, roles, transacciones o navegación principal.

14.6 Recomendaciones de calidad para el repositorio

- Mantener un README vivo con instrucciones de instalación, configuración, ejecución, usuarios de prueba, tecnologías utilizadas y estado del proyecto.
- No subir contraseñas reales, llaves privadas, tokens, datos sensibles o información personal de clientes reales.
- Utilizar archivos de ejemplo para variables de entorno cuando se requieran credenciales o configuraciones locales.
- Registrar tareas pendientes o incidencias en GitHub Issues, Trello, Planner u otra herramienta acordada por el equipo.
- Probar el sistema antes de cada entrega con usuarios y roles distintos.
- Preparar una base de datos de demostración con datos ficticios, coherentes y suficientes.

15. Lista de verificación por entrega

Momento	Lista de verificación
Semana 5	20 historias de usuario; criterios de aceptación; backlog priorizado; prototipo; mapa de navegación; entidades preliminares; presentación en vídeo con participación total.
Semana 9	Proyecto Spring Boot funcional; Bootstrap aplicado; conexión a base de datos; CRUD principal; 50% del alcance implementado; GitHub con aportes de todos; README preliminar.
Semana 15 - artículo	Artículo IEEE de 6 a 12 páginas; resumen, introducción, metodología, resultados, discusión, conclusiones y referencias; mínimo 10 referencias pertinentes.
Semana 15 - entrega final	Aplicación completa; 8 tablas; tabla transaccional; roles funcionales; internacionalización; 6 temas del curso; investigación adicional; GitHub final; base de datos; usuarios de prueba.
Semana 15 - defensa	Demostración funcional; explicación técnica; navegación por artículo; evidencia de colaboración; participación equitativa; respuestas a preguntas del docente.

16. Formato de entrega sugerido

Cada entrega debe subirse en el espacio habilitado por el docente en Moodle y en el canal privado de cada equipo en Teams. Se recomienda entregar un archivo PDF o documento con enlaces activos y una carpeta o repositorio con los productos técnicos.

- Enlace al repositorio GitHub.
- Enlace al prototipo o archivo exportado.
- Documento PDF del avance o artículo.
- Video corto de demostración.
- Script o respaldo de base de datos.
- Usuarios de prueba por rol, sin contraseñas personales reales.
- Indicaciones especiales de instalación o despliegue.

17. Honestidad académica y uso responsable de apoyo externo

El proyecto debe evidenciar trabajo original del equipo. Se permite consultar documentación, ejemplos, bibliografía y recursos técnicos, siempre que se comprendan, adapten y citen cuando corresponda. No se permite presentar como propio código, diseño, documentación o artículos elaborados por terceros.

El uso de herramientas de apoyo para depuración, redacción o consulta técnica no sustituye la autoría ni el dominio del equipo. Durante la defensa, cada integrante debe explicar las partes que desarrolló y justificar las decisiones tomadas.

18. Cierre

Este enunciado busca orientar la construcción del proyecto final como una experiencia integradora, aplicada y evaluable. La calidad del proyecto dependerá de la coherencia entre la necesidad planteada, las historias de usuario, el prototipo, la implementación técnica, la documentación científica y la defensa oral.

El docente podrá ajustar detalles operativos, herramientas de entrega o énfasis de evaluación de acuerdo con las condiciones del grupo, siempre que se mantengan los criterios esenciales del programa del curso.